

Ros2 Day2 hw1 보고서

휴머노이드 수습단원 이주원

목차

1. 프로젝트 개요
2. 코드 구조
3. 파일 구조
4. 실행결과

1. 프로젝트 개요

이 프로젝트는 ROS2에서 사용자 정의 메시지를 만들어 두 노드가 정수 배열을 주고받는다. 기본으로 제공되는 메시지 대신 직접 만든 메시지를 사용하며, 퍼블리셔와 서브스크라이버를 C++로 구현한다.

메시지는 따로 만든 인터페이스 패키지에 정의했다. 정수 하나와 정수 배열, 두 개의 필드를 가지고 있는데 이번 과제에서는 길이가 정해져 있지 않은 배열 쪽만 사용한다. 토픽 이름은 `my_vector_topic`으로 정했고 큐 사이즈는 10으로 두었다.

퍼블리셔는 터미널에서 값을 직접 입력받는다. 먼저 보낼 정수가 몇 개인지 입력받고, 그 개수만큼 값을 하나씩 입력받아 벡터에 저장한 뒤 토픽으로 발행한다. 발행이 끝나면 벡터를 비우고 다시 입력을 기다리기 때문에 프로그램을 끄기 전까지 계속 반복해서 보낼 수 있다.

개수에 음수가 들어오면 값을 받지 않고 넘어가도록 해놓았다.

서브스크라이버는 같은 토픽을 구독한다. 메시지가 들어오면 받은 배열의 크기를 먼저 출력하고, 반복문을 돌면서 각 원소를 인덱스와 함께 터미널에 출력한다.

두 노드 모두 클래스로 작성했고 헤더와 소스 파일을 나누어 구성했다.

2. 코드 구조

```
MyPublisher::MyPublisher() : Node("my_publisher")
```

```
{  
    pub_ = this->create_publisher<custom_msg_interface::msg::VectorMsg>("my_vector_topic",  
    10);  
}
```

노드를 my_publisher라는 이름으로 초기화하고, my_vector_topic 토픽에 커스텀 메시지를 발행할 퍼블리셔를 만들어 멤버로 저장하는 부분

```
void MyPublisher::input_data()
```

이전에 담아둔 값을 먼저 비우고, 보낼 정수가 몇 개인지 입력받은 뒤 그 개수만큼 반복하며 값을 하나씩 벡터에 넣는 부분. 개수가 음수로 들어오면 값을 받지 않고 그냥 빠져나온다.

```
void MyPublisher::publish_data()
```

```
{  
    auto msg = custom_msg_interface::msg::VectorMsg();  
    msg.b = vec_;  
    pub_->publish(msg);  
    RCLCPP_INFO(this->get_logger(), "Published vector message with %zu elements.",  
vec_.size());  
}
```

메시지 객체를 만들어 배열 필드에 벡터를 그대로 넣고 발행한 뒤, 몇 개를 보냈는지 로그로 남기는 부분

```
int main(int argc, char **argv)
```

노드를 만들고 입력받기와 발행하기를 계속 반복하는 메인 루프. 서브스크라이버와 달리 스핀을 돌지 않고 입력을 기다리는 구조라서 이렇게 작성했다.

```
MySubscriber::MySubscriber() : Node("my_subscriber")
```

```
{
```

```
sub_=this->create_subscription<custom_msg_interface::msg::VectorMsg>("my_vector_topic",1  
0,std::bind(&MySubscriber::vector_callback,this,std::placeholders::_1));
```

```
}
```

노드를 my_subscriber라는 이름으로 초기화하고, 퍼블리셔와 같은 토픽을 구독하면서 메시지가 들어왔을 때 실행될 콜백 함수를 연결하는 부분

```
void MySubscriber::vector_callback(const custom_msg_interface::msg::VectorMsg::SharedPtr  
msg)
```

```
{
```

```
    RCLCPP_INFO(this->get_logger(), "Received message: '%zu'", msg->b.size());
```

```
    for (size_t i=0; i < msg->b.size(); i++)
```

```
    {
```

```
        RCLCPP_INFO(this->get_logger(), "int32_b[%zu] = %d", i, msg->b[i]);
```

```
    }
```

```
}
```

메시지가 들어오면 받은 배열의 크기를 먼저 출력하고, 반복문을 돌면서 각 원소를 인덱스와 함께 출력하는 부분

```
int64 a
```

```
int32[] b
```

직접 정의한 메시지 파일. 정수 하나와 정수 배열을 가지고 있고, 이번 과제에서는 배열만 사용한다.

3. 파일 구조

```
hw1
```

```
|—— build
```

```
|—— install
```

```
|—— log
```

```
|—— src
```

```
    |—— custom_msg_interface
```

```
        |—— include
```

```
|   |—— msg
|   |   └── VectorMsg.msg
|   |—— src
|   |—— CMakeLists.txt
|   └── package.xml
|—— juwon_topic
|   |—— include
|   |   |—— my_publisher.hpp
|   |   └── my_subscriber.hpp
|   |—— src
|   |   |—— my_publisher.cpp
|   |   └── my_subscriber.cpp
|   |—— CMakeLists.txt
|   └── package.xml
└── hw1.mp4
```

juwon_topic에는 실제 노드 코드가 들어 있다. 클래스 선언은 include 폴더의 헤더에, 구현과 메인 함수는 src 폴더의 소스 파일에 나누어 두었다.

4. 실행결과

첨부 영상 참고